

HDL Design Homework: 4-Bit Adder Architectures

Yousef Awad
EEL 4783 - HDL in Digital Systems

March 9, 2026

Part 1: Design Specifications

Below is the synthesizable HDL code for the Ripple-Carry Adder (RCA), Carry-Lookahead Adder (CLA), and the associated testbench.

D. Full Adder

File: full_adder.sv

```
1 `timescale 1ns / 10ps
2
3 module full_adder (
4     input  a,
5     input  b,
6     input  c_in,
7     output c_out,
8     output sum
9 );
10
11 assign #1 sum    = a ^ b ^ c_in;
12 assign #1 c_out = (a & b) | (b & c_in) | (a & c_in);
13
14 endmodule
```

B. Ripple-Carry Adder (RCA)

File: rca.sv

```
1 `timescale 1ns / 10ps
2
3 module rca (
4     input [3:0] A,
5     input [3:0] B,
6     input  Cin,
7     output Cout,
8     output [3:0] S
9 );
10
11 genvar i;
12 wire [4:0] carry_temp; // propagate
13
14 // b0 is lsb, for context
15 generate
```

```

16     for (i = 0; i < 4; i = i + 1) begin : g_adder_loop
17         full_adder adder (
18             .a(A[i]),
19             .b(B[i]),
20             .c_in(carry_temp[i]),
21             .c_out(carry_temp[i+1]),
22             .sum(S[i])
23         );
24     end
25 endgenerate
26
27 assign Cout = carry_temp[4]; // the c_out is always the MSB of the temp_carry
28 assign carry_temp[0] = Cin;
29
30 endmodule

```

C. Carry-Lookahead Adder (CLA)

File: cla.sv

```

1  `timescale 1ns / 10ps
2
3  module cla (
4      input [3:0] A,
5      input [3:0] B,
6      input Cin,
7      output Cout,
8      output [3:0] S
9  );
10
11  // create the generate and propagate and buffers for carries
12  wire [3:0] Generate;
13  wire [3:0] Propagate;
14  wire temp_carry[2:0];
15
16  // define the logic for the generate and propagate
17  assign Generate = A & B;
18  assign Propagate = A ^ B;
19
20  // define the logic for the buffer carries and output carry
21  assign temp_carry[0] = Generate[0] | (Propagate[0] & Cin);
22  assign temp_carry[1] = Generate[1] | (Propagate[1] & Generate[0]) | (Propagate
    [1] & Propagate[0] & Cin);
23  assign temp_carry[2] = Generate[2] | (Propagate[2] & Generate[1]) | (Propagate
    [2] & Propagate[1] & Generate[0]) | (Propagate[2] & Propagate[1] & Propagate[0]
    & Cin);
24  assign Cout = Generate[3] | (Propagate[3] & Generate[2]) | (Propagate[3] &
    Propagate[2] & Generate[1]) |
25  (Propagate[3] & Propagate[2] & Propagate[1] & Generate[0]) | (Propagate[3] &
    Propagate[2] & Propagate[1] & Propagate[0] & Cin);
26
27  // defining the summation logic
28  assign S[0] = Propagate[0] ^ Cin;
29  assign S[1] = Propagate[1] ^ temp_carry[0];
30  assign S[2] = Propagate[2] ^ temp_carry[1];
31  assign S[3] = Propagate[3] ^ temp_carry[2];
32
33 endmodule

```

D. Testbench

File: testbench_adders.sv

```
1 `timescale 1ns / 10ps
2
3 module testbench_adders ();
4
5     // Inputs
6     reg [3:0] A, B;
7     reg Cin;
8
9     // Outputs
10    wire [3:0] out_rca, out_cla;
11    wire cout_rca, cout_cla;
12
13    cla cla_adder (
14        .A(A),
15        .B(B),
16        .Cin(Cin),
17        .Cout(cout_cla),
18        .S(out_cla)
19    );
20    rca rca_adder (
21        .A(A),
22        .B(B),
23        .Cin(Cin),
24        .Cout(cout_rca),
25        .S(out_rca)
26    );
27
28    initial begin
29        $fsdbDumpfile("dump.fsdb");
30        $fsdbDumpvars(0, testbench_adders);
31
32        A    = 'b0;
33        B    = 'b0;
34        Cin  = 'b0;
35        #10;
36        assert ({cout_cla, out_cla} == (A + B + Cin)) else $display("UHOH SPEGHATTI");
37        assert ({cout_rca, out_rca} == (A + B + Cin)) else $display("UHOH SPEGHATTI");
38
39        A    = 'b1111;
40        B    = 'b0001;
41        Cin  = 'b0;
42        #10;
43        assert ({cout_cla, out_cla} == (A + B + Cin)) else $display("UHOH SPEGHATTI");
44        assert ({cout_rca, out_rca} == (A + B + Cin)) else $display("UHOH SPEGHATTI");
45
46        A    = 'b1010;
47        B    = 'b0101;
48        Cin  = 'b1;
49        #10;
50        assert ({cout_cla, out_cla} == (A + B + Cin)) else $display("UHOH SPEGHATTI");
51        assert ({cout_rca, out_rca} == (A + B + Cin)) else $display("UHOH SPEGHATTI");
52
53        A    = 'b1111;
54        B    = 'b1111;
55        Cin  = 'b1;
```

```

56     #10;
57     assert ({cout_cla, out_cla} == (A + B + Cin)) else $display("UHOH SPEGHATTI");
58     assert ({cout_rca, out_rca} == (A + B + Cin)) else $display("UHOH SPEGHATTI");
59
60     A    = 'b0111;
61     B    = 'b0001;
62     Cin = 'b0;
63     #10;
64     assert ({cout_cla, out_cla} == (A + B + Cin)) else $display("UHOH SPEGHATTI");
65     assert ({cout_rca, out_rca} == (A + B + Cin)) else $display("UHOH SPEGHATTI");
66
67     #50;
68
69     $finish;
70 end
71
72 endmodule

```

Part 2: Testbench & Simulation

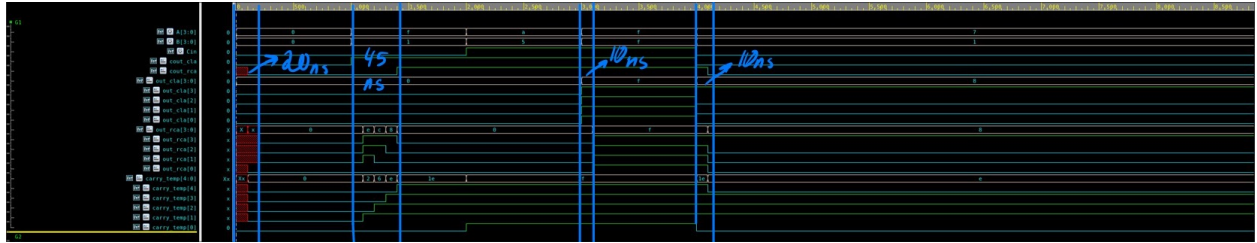


Figure 1: **Functional Comparison.** The waveforms confirm that both RCA and CLA produce identical Sum and Carry-out values for all test vectors.

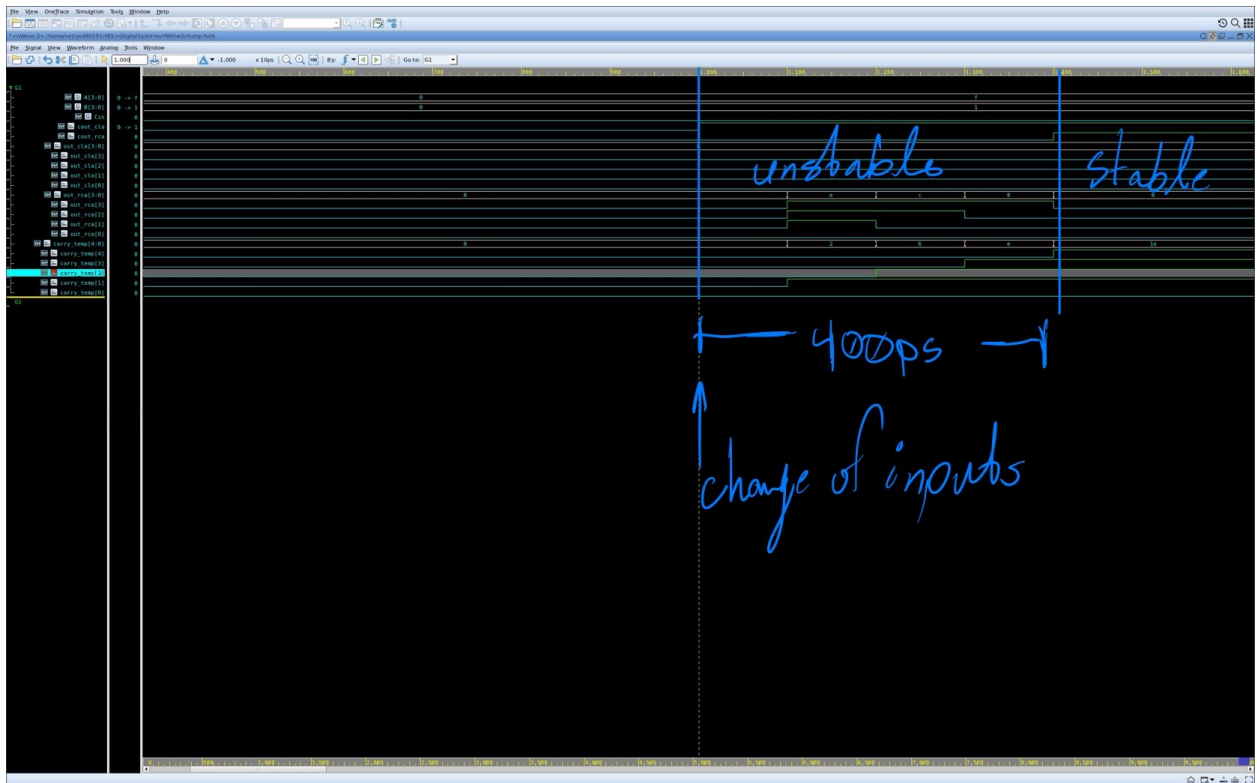


Figure 2: **Critical Path Zoom-In.** The timing detail reveals the “ripple” effect in the RCA carry chain ($C_1 \rightarrow C_2 \rightarrow C_3$), while the CLA output settles significantly faster.

Part 3: Analysis & Discussion

1. Timing Comparison

For test vector #5 (Critical Path Test), the propagation delays were measured from the input change to stable C_{out} .

- **RCA Delay:** ≈ 400 ps (approximate, see Fig 2)
- **CLA Delay:** Negligible / Instantaneous relative to RCA

Explanation: The Ripple-Carry Adder exhibits sequential propagation because the carry-out of bit i (C_{i+1}) depends directly on the carry-in from the previous bit (C_i). This creates a dependency chain where the signal must traverse every Full Adder physically. In contrast, the CLA calculates all carry bits simultaneously using flattened logic equations based purely on inputs A , B , and C_{in} , eliminating the “waiting” period.

2. Circuit Explanation

At the gate level, the RCA delay scales as $O(N)$ because the critical path flows through N Full Adder blocks in series. If a single Full Adder has a delay of t_{FA} , the total delay is $N \times t_{FA}$. The CLA delay scales as $O(\log N)$ (or effectively $O(1)$ for fixed logic depths). This is because the carry logic is expanded into a Sum-of-Products form. For example, C_4 is calculated as:

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_{in}$$

This equation is realized with a 2-level AND-OR logic depth regardless of the bit position (limited only by the fan-in capability of the gates), allowing simultaneous calculation.

3. Area-Speed Tradeoff

The CLA requires significantly more logic gates than the RCA.

- **RCA:** Uses ≈ 5 gates per bit. Total for 4-bit ≈ 20 gates. Linear area growth.
- **CLA:** Requires complex logic for the Generate/Propagate lookahead units. The gate count grows cubically/exponentially depending on implementation details.

Designers might still choose RCA for wide adders (e.g., >16 bits) if the application is **low-power** or **area-constrained** and the adder is not on the critical timing path. CLA is preferred strictly when minimizing delay is the highest priority.

4. Real-World Relevance

Modern FPGAs (like Xilinx 7-Series or Ultrascale) contain dedicated hardware resources known as **Carry Chains** (e.g., CARRY4 or CARRY8 primitives). These are hardened, high-speed silicon paths located adjacent to the LUTs. This hardware optimization allows Ripple-Carry structures to operate at extremely high speeds, often mitigating the theoretical delay disadvantage of RCA and negating the need for complex soft-logic CLAs for standard integer addition.

Extra Credit: Synthesis Report Analysis

Synthesis reports were generated for both designs to quantify the hardware cost. The reports below compare the combinatorial area and leaf cell count for the 4-bit implementations.

Metric	RCA	CLA	Difference
Combinational Area	94.00	98.00	+4.2%
Leaf Cell Count	54	57	+3 Cells
Sequential Cells (FFs)	0	0	0

Table 1: **Synthesis Quality of Results (QoR) Comparison.** The CLA consumes slightly more area due to the additional lookahead logic gates.

Analysis: The synthesis results confirm the theoretical area-speed tradeoff. The RCA is smaller (Area: 94.00) because it uses a simple chain of full adders. The CLA is larger (Area: 98.00) because of the extra logic required for the Generate/Propagate and Carry Lookahead units. Both designs are purely combinational, resulting in zero sequential cells (Flip-Flops).

Synthesis Report Snippets

RCA Quality of Results (excerpt):

```
1  Cell Count
2  -----
3  Leaf Cell Count:          54
4  Combinational Cell Count: 54
5  Sequential Cell Count:    0
6  -----
7
8  Area
9  -----
10 Combinational Area:      94.000000
11 Buf/Inv Area:           38.000000
12 Cell Area:               94.000000
```

CLA Quality of Results (excerpt):

```
1  Cell Count
2  -----
3  Leaf Cell Count:          57
4  Combinational Cell Count: 57
5  Sequential Cell Count:    0
6  -----
7
8  Area
9  -----
10 Combinational Area:      98.000000
11 Buf/Inv Area:           41.000000
12 Cell Area:               98.000000
```